

```

using System;
using UnityEngine;
using UnityEngine.Events;

[Serializable]
public sealed class GrabbableEvent :
    UnityEvent<Object_Grabbable>
{
}

[DisallowMultipleComponent]
public sealed class InventoryUIManager :
    MonoBehaviour
{
    [Header("Core")]
    [SerializeField]
    private InventoryData inventoryData;

    [SerializeField]
    private PerceiveObjectHandPivot
handPerception;

    [SerializeField]
    private BringData bringData;

    [SerializeField]
    private InventoryUIEffect uiEffect;

    [Header("Window")]
    [SerializeField]
    private bool startClosed = true;

    [Header("인벤토리 중 게임 입력")]
    [Tooltip("인벤토리가 열렸을 때 이동/시점 입력 스
크립트")]
    [SerializeField]
    private MonoBehaviour[] disableWhileOpen;

    [SerializeField]
    private bool pauseGameWhileOpen;

    [Header("Optional Equip Request")]
    [SerializeField]
    private GrabbableEvent onEquipRequested;

    [Header("Debug")]
    [SerializeField]
    private bool showDebugLog = true;

```

```

private Object_Grabbable currentHeldObject;
private InventoryDisplayData
selectedDisplayData;
private bool isOpen;

private CursorLockMode previousCursorLock;
private bool previousCursorVisible;
private bool[] previousBehaviourStates;
private float previousTimeScale = 1f;
private bool inventoryControlModeApplied;

public bool IsOpen => isOpen;

public Object_Grabbable CurrentHeldObject =>
    currentHeldObject;

public InventoryDisplayData
SelectedDisplayData =>
    selectedDisplayData;

private void Awake()
{
    isOpen = !startClosed;

    if (uiEffect != null)
    {
        uiEffect.BindSlots(
            SelectSlot
        );

        uiEffect.SetOpen(
            isOpen
        );
    }
}

private void OnEnable()
{
    if (inventoryData != null)
    {
        inventoryData.Changed +=
            HandleInventoryChanged;

        inventoryData.InventoryFull +=
            HandleInventoryFull;
    }

    if (handPerception != null)
    {

```

```

        handPerception.HandObjectChanged +=
            HandleHandObjectChanged;
    }
}

private void Start()
{
    if (isOpen)
    {
        EnterInventoryControlMode();
    }

    if (handPerception != null)
    {
        handPerception.ForceScan();
    }

    SyncSelectedDisplayData();
    RefreshView();
}

private void LateUpdate()
{
    if (!isOpen)
    {
        return;
    }

    Cursor.lockState =
        CursorLockMode.None;

    Cursor.visible = true;
}

private void OnDisable()
{
    if (handPerception != null)
    {
        handPerception.HandObjectChanged -=
            HandleHandObjectChanged;
    }

    if (inventoryData != null)
    {
        inventoryData.Changed -=
            HandleInventoryChanged;

        inventoryData.InventoryFull -=
            HandleInventoryFull;
    }
}

    if (uiEffect != null)
    {
        uiEffect.SetOpen(false);
    }

    ExitInventoryControlMode();
}

public void ToggleInventory()
{
    if (isOpen)
    {
        CloseInventory();
    }
    else
    {
        OpenInventory();
    }
}

public void OpenInventory()
{
    if (isOpen)
    {
        return;
    }

    isOpen = true;

    EnterInventoryControlMode();

    if (uiEffect != null)
    {
        uiEffect.SetOpen(true);
    }

    if (handPerception != null)
    {
        handPerception.ForceScan();
    }

    SyncSelectedDisplayData();
    RefreshView();
}

public void CloseInventory()
{

```

```

        if (!isOpen)
        {
            return;
        }

        isOpen = false;

        if (uiEffect != null)
        {
            uiEffect.SetOpen(false);
        }

        SyncSelectedDisplayData();
        RefreshView();

        ExitInventoryControlMode();
    }

    public void SelectSlot(
        int slotIndex)
    {
        if (inventoryData == null)
        {
            Debug.LogError(
                "[ I n v e n t o r y U I M a n a g e r ]
InventoryData가 없습니다.",
                this
            );

            return;
        }

        if (showDebugLog)
        {
            Debug.Log(
                $"[InventoryUIManager] 슬롯 선택
요청: {slotIndex}",
                this
            );
        }

        inventoryData.Select(
            slotIndex
        );

        SyncSelectedDisplayData();
        RefreshView();

        if (showDebugLog)
    {
        string selectedName =
            selectedDisplayData != null
                ?
                selectedDisplayData.ItemName
                : "null";

        Debug.Log(
            $"[InventoryUIManager] 선택 결과: "
            +
            $"SelectedIndex={inventoryData.SelectedIndex}, " +
            $"EquippedIndex={inventoryData.EquippedIndex}, " +
            $"SelectedItem={selectedName}",
            this
        );
    }
    }

    public void ConfirmSelectedItem()
    {
        if (inventoryData == null)
        {
            return;
        }

        Object_Grabbable selectedObject =
            inventoryData.SelectedObject;

        if (selectedObject == null)
        {
            return;
        }

        onEquipRequested?.Invoke(
            selectedObject
        );
    }

    private void HandleHandObjectChanged(
        Object_Grabbable detectedObject)
    {
        if (inventoryData == null)
        {
            Debug.LogError(
                "[ I n v e n t o r y U I M a n a g e r ]
InventoryData가 없습니다.",
                this
            );
        }
    }

```

```

    );

    return;
}

currentHeldObject =
    detectedObject;

if (showDebugLog)
{
    string detectedName =
        detectedObject != null
            ?
detectedObject.gameObject.name
            : "null";

    string objectName =
        detectedObject != null
            ?
ResolveObjectName(detectedObject)
            : "null";

    Debug.Log(
        $"[InventoryUIManager]
HandObjectChanged: " +
        $"Source={detectedName}, " +
        $"ObjectName={objectName}",
        this
    );
}

/*
* HandPivot이 비었으면 장착 상태만 해제합
니다.

* 슬롯 삭제는 Release/UseKey/PutOn에서
* InventoryData.RemoveBySource()로 처리
합니다.
*/
if (currentHeldObject == null)
{
    inventoryData.SetEquipped(-1);

    SyncSelectedDisplayData();
    RefreshView();
    return;
}

/*
* 같은 이름이더라도 실제 SourceObject가 다

```

```

르면

    * FindIndexBySource() 결과는 -1이므로 새
슬롯에 들어갑니다.
    */
    int index =
        inventoryData.FindIndexBySource(
            currentHeldObject
        );

    if (index < 0)
    {
        /*
        * InventoryData가 index 0부터 첫 빈
슬롯을 찾고,
        * Object_Grabbable.objectName을 저장
합니다.
        */
        if (!inventoryData.TryAdd(
            currentHeldObject,
            out index))
        {
            SyncSelectedDisplayData();
            RefreshView();
            return;
        }
    }

    inventoryData.SetSelectedAndEquipped(
        index
    );

    SyncSelectedDisplayData();
    RefreshView();

    if (showDebugLog)
    {
        Debug.Log(
            $"[InventoryUIManager] 장착 완료: "
+
            $"Index={index}, " +

            $"SelectedIndex={inventoryData.SelectedIndex}, " +

            $"EquippedIndex={inventoryData.EquippedIndex}, " +

            $"ObjectName={inventoryData.GetObjectNameAt(index)}, " +

            $"Source={currentHeldObject.gameObject.name}",

```

```

        currentHeldObject
    );
}
}

private void HandleInventoryChanged()
{
    SyncSelectedDisplayData();
    RefreshView();
}

private void HandleInventoryFull()
{
    if (uiEffect != null)
    {
        uiEffect.PlayInventoryFull();
    }
}

private void SyncSelectedDisplayData()
{
    if (inventoryData == null ||
        bringData == null)
    {
        selectedDisplayData = null;
        return;
    }

    selectedDisplayData =
        bringData.BuildDisplayData(
            inventoryData,
            inventoryData.SelectedIndex
        );
}

private void RefreshView()
{
    if (uiEffect == null)
    {
        return;
    }

    uiEffect.Refresh(
        inventoryData,
        selectedDisplayData,
        isOpen
    );
}

```

```

private static string ResolveObjectName(
    Object_Grabbable sourceObject)
{
    if (sourceObject == null)
    {
        return "Unknown";
    }

    if (!string.IsNullOrEmpty(
        sourceObject.objectName))
    {
        return sourceObject.objectName;
    }

    return sourceObject.gameObject.name;
}

private void EnterInventoryControlMode()
{
    if (inventoryControlModeApplied)
    {
        return;
    }

    inventoryControlModeApplied = true;

    previousCursorLock =
        Cursor.lockState;

    previousCursorVisible =
        Cursor.visible;

    Cursor.lockState =
        CursorLockMode.None;

    Cursor.visible = true;

    if (pauseGameWhileOpen)
    {
        previousTimeScale =
            Time.timeScale;

        Time.timeScale = 0f;
    }

    if (disableWhileOpen == null)
    {
        return;
    }
}

```

```

previousBehaviourStates =
    new bool[disableWhileOpen.Length];

for (int i = 0;
    i < disableWhileOpen.Length;
    i++)
{
    MonoBehaviour behaviour =
        disableWhileOpen[i];

    if (behaviour == null ||
        behaviour == this)
    {
        continue;
    }

    previousBehaviourStates[i] =
        behaviour.enabled;

    behaviour.enabled = false;
}

private void ExitInventoryControlMode()
{
    if (!inventoryControlModeApplied)
    {
        return;
    }

    inventoryControlModeApplied = false;

    if (pauseGameWhileOpen)
    {
        Time.timeScale =
            previousTimeScale;
    }

    if (disableWhileOpen != null &&
        previousBehaviourStates != null)
    {
        int count =
            Mathf.Min(
                disableWhileOpen.Length,
                previousBehaviourStates.Length
            );

        for (int i = 0;
            i < count;
            i++)
        {
            MonoBehaviour behaviour =
                disableWhileOpen[i];

            if (behaviour == null ||
                behaviour == this)
            {
                continue;
            }

            behaviour.enabled =
                previousBehaviourStates[i];
        }

        previousBehaviourStates = null;

        Cursor.lockState =
            previousCursorLock;

        Cursor.visible =
            previousCursorVisible;
    }
}

```